



Gobernanza y sistemas de votación

**TECNOLOGÍA BLOCKCHAIN
Y SMART CONTRACTS**

ALEJANDRO SERRANO RUIBAL



Contenido

1. Introducción y objetivo del proyecto	2
2. Arquitectura general del sistema	2
3. Diseño pull-over-push	3
4. Pruebas realizadas	4
4.1. Despliegue de QuadraticVoting y obtención del ERC20	4
4.2. Registro de participantes	5
4.3. Compra, venta y eliminación de participante	6
4.4. Apertura de votación	8
4.5. Creación de propuesta inválida	9
4.6. Creación de propuesta de financiación válida	10
4.7. Prueba negativa: votar sin approve	10
4.8. Aprobación de tokens y votación con stake	11
4.9. Aprobación automática de una propuesta de financiación	12
4.10. Retirada de votos	13
4.11. Cancelación de propuesta y reembolso	14
4.12. Creación y votación de propuesta de signaling	15
4.13. Cierre de votación	17
4.14. Ejecución de propuesta de signaling después del cierre	17
4.15. Reclamación de tokens de propuesta no aprobada	18
4.16. Reclamación de tokens de propuesta de signaling	18
4.17. Prueba negativa: no se puede reclamar en propuesta aprobada	19
5. Seguridad y análisis de vulnerabilidades	19
5.1. Protección frente a reentrancy	20
5.2. Contratos externos maliciosos	21
5.3. Denegación de servicio por consumo de gas	21
5.4. Protección de los tokens ERC20	22
5.5. Transferencias de Ether con call	22
5.6. Control de permisos y validación de estados	22
5.7. Tamaño del bytecode	22
5.8. Conclusión del análisis de seguridad	23
6. Conclusiones	23

1. Introducción y objetivo del proyecto

El presente proyecto consiste en el desarrollo de un sistema de votación cuadrática on-chain para DAOs mediante smart contracts en Solidity. El objetivo es permitir que distintos participantes puedan registrarse, comprar tokens de votación, crear propuestas y votar sobre ellas siguiendo un modelo en el que el coste de los votos crece de forma cuadrática. Así, un voto cuesta 1 token, dos votos cuestan 4 tokens, tres votos cuestan 9 tokens, etc. De esta forma, los participantes pueden mostrar una preferencia más fuerte por ciertas propuestas, pero evitando que concentrar muchos votos resulte demasiado barato.

El sistema distingue entre dos tipos de propuestas. Las propuestas de financiación solicitan una cantidad concreta de Ether del presupuesto de la votación y pueden aprobarse automáticamente si existe presupuesto suficiente y superan el umbral de votos establecido. Por otro lado, las propuestas de signaling no tienen presupuesto asociado, sino que sirven para expresar preferencias generales de la DAO. Estas propuestas se votan durante la ronda, pero se ejecutan después del cierre de la votación.

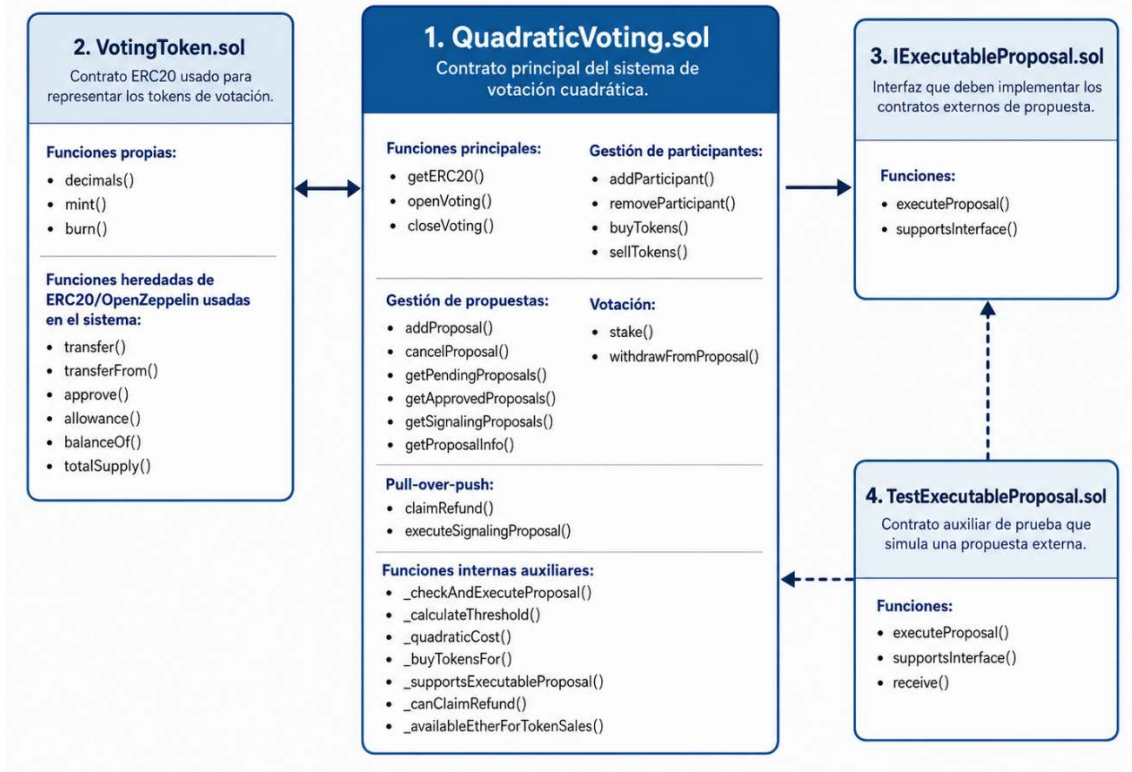
Además, se ha incorporado el patrón pull-over-push, de forma que al cerrar una votación no se recorren todos los votantes ni todas las propuestas. En su lugar, cada usuario puede reclamar sus tokens mediante `claimRefund`, y las propuestas de signaling pueden ejecutarse individualmente mediante `executeSignalingProposal`. Este diseño reduce el riesgo de problemas de gas y mejora la escalabilidad del contrato.

Finalmente, el sistema se ha diseñado teniendo en cuenta la seguridad, ya que maneja Ether y tokens con valor económico. Para ello se utilizan mecanismos como `approve` y `transferFrom`, verificación de contratos externos mediante ERC165, transferencias de Ether con `call`, límite de gas en ejecuciones externas y protección frente a reentrancy.

2. Arquitectura general del sistema

El sistema se ha dividido en varios contratos para separar responsabilidades y mantener una estructura clara. El contrato central es `QuadraticVoting`, que gestiona la lógica de votación. El contrato `VotingToken` representa los tokens ERC20 utilizados para votar. La interfaz `IExecutableProposal` define cómo debe ejecutarse cualquier propuesta externa. Finalmente, `TestExecutableProposal` se utiliza como contrato auxiliar de prueba para comprobar que las propuestas externas se ejecutan correctamente.

ARQUITECTURA GENERAL DEL SISTEMA



3. Diseño pull-over-push

El enunciado advierte que `closeVoting()` puede llegar a consumir demasiado gas si intenta realizar todas las tareas de cierre en una única transacción. En un diseño tradicional, esta función tendría que recorrer todas las propuestas y todos los votantes para devolver tokens, ejecutar las propuestas de signaling y transferir el presupuesto sobrante. Este enfoque no es adecuado en un sistema de gobernanza, ya que el número de participantes y propuestas puede crecer con el tiempo. Si estas estructuras fueran demasiado grandes, `closeVoting()` podría superar el límite de gas y quedar bloqueada, provocando una denegación de servicio.

Para evitar este problema, se ha integrado el patrón pull-over-push. En lugar de hacer que el contrato envíe automáticamente todos los reembolsos y ejecute todas las propuestas de signaling dentro de `closeVoting()`, se permite que estas operaciones se realicen posteriormente y de forma individual. De esta manera, `closeVoting()` queda limitada a cerrar la ronda, poner a cero el presupuesto activo, transferir al owner el presupuesto no gastado y emitir el evento `VotingClosed`.

La devolución de tokens se realiza mediante `claimRefund()`. Esta función permite que cada usuario reclame los tokens que tenía bloqueados en una propuesta concreta. Se puede utilizar cuando la propuesta ha sido cancelada, cuando una propuesta de financiación no ha sido aprobada al cerrarse la votación o cuando se trata de una propuesta de signaling después del cierre. En cambio, no se permite

reclamar tokens de una propuesta de financiación aprobada, ya que en ese caso los tokens se consumen como parte del proceso de aprobación. Para evitar dobles devoluciones, el contrato utiliza el mapping `refundClaimed[proposalId][participant]`.

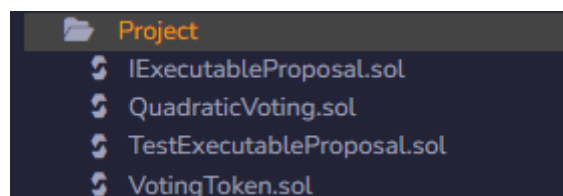
Las propuestas de signaling también se gestionan bajo demanda. En vez de ejecutarse automáticamente dentro de `closeVoting()`, se ejecutan después del cierre mediante `executeSignalingProposal()`. Esta función no envía Ether, porque las propuestas de signaling tienen presupuesto cero, y simplemente ejecuta el contrato externo asociado para dejar constancia del resultado consultivo. Además, el campo `signalingExecuted` impide que una misma propuesta de signaling pueda ejecutarse más de una vez.

Con este diseño, el cierre de la votación no depende de recorrer todos los usuarios ni todas las propuestas. Esto reduce el riesgo de problemas de gas, permite que cada participante reclame sus tokens de forma individual y mantiene el contrato preparado para abrir nuevas rondas de votación sin dejar operaciones pendientes bloqueadas dentro de `closeVoting()`.

4. Pruebas realizadas

Las pruebas del sistema se realizaron en Remix utilizando distintas cuentas de la máquina virtual para simular los roles de owner, participantes y contratos externos de propuesta. El objetivo de esta fase fue comprobar que los contratos se desplegaban correctamente, que la gestión de participantes y tokens funcionaba según lo esperado, que las propuestas podían crearse y votarse correctamente, y que el sistema respondía de forma adecuada tanto en casos válidos como en situaciones que debían provocar un rechazo de la transacción.

Además de comprobar el funcionamiento individual de cada función, se realizaron pruebas conjuntas que reproducen un ciclo completo de votación: despliegue del sistema, registro de participantes, apertura de ronda, creación de propuestas, votación mediante tokens ERC20, aprobación automática de propuestas de financiación, cierre de la votación, ejecución de propuestas de signaling y reclamación de tokens mediante el patrón pull-over-push.



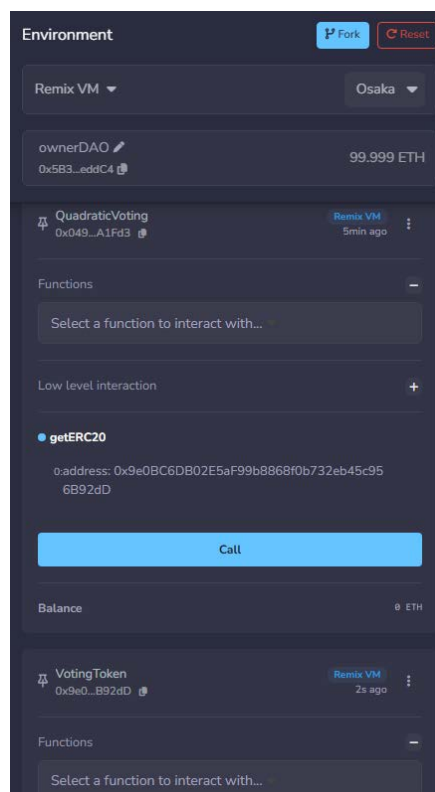
4.1. Despliegue de QuadraticVoting y obtención del ERC20

En primer lugar, se desplegó el contrato `QuadraticVoting`, proporcionando en el constructor el precio de cada token en Wei y el número máximo de tokens

disponibles para el sistema. Al desplegar este contrato, se crea automáticamente una instancia de VotingToken, que será el token ERC20 utilizado por los participantes para votar.

Tras el despliegue, se ejecutó la función `getERC20()`, que devolvió la dirección del contrato VotingToken creado por QuadraticVoting. Con esta dirección, se abrió el contrato VotingToken en Remix mediante la opción Add Contract, sin desplegar un nuevo token manualmente. Esta comprobación es importante porque el sistema solo debe operar con el token creado internamente por QuadraticVoting.

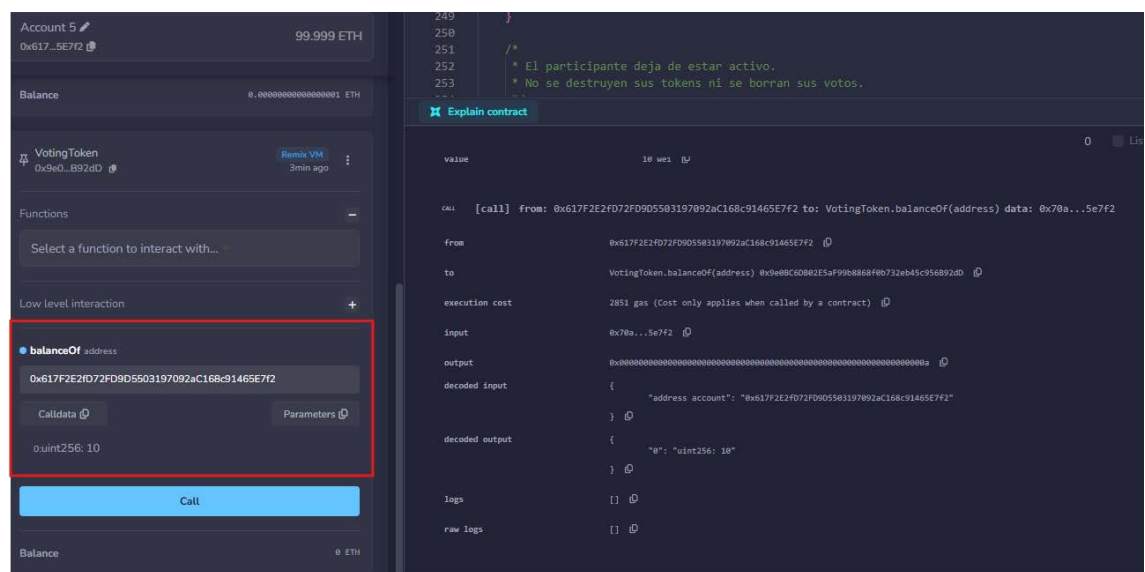
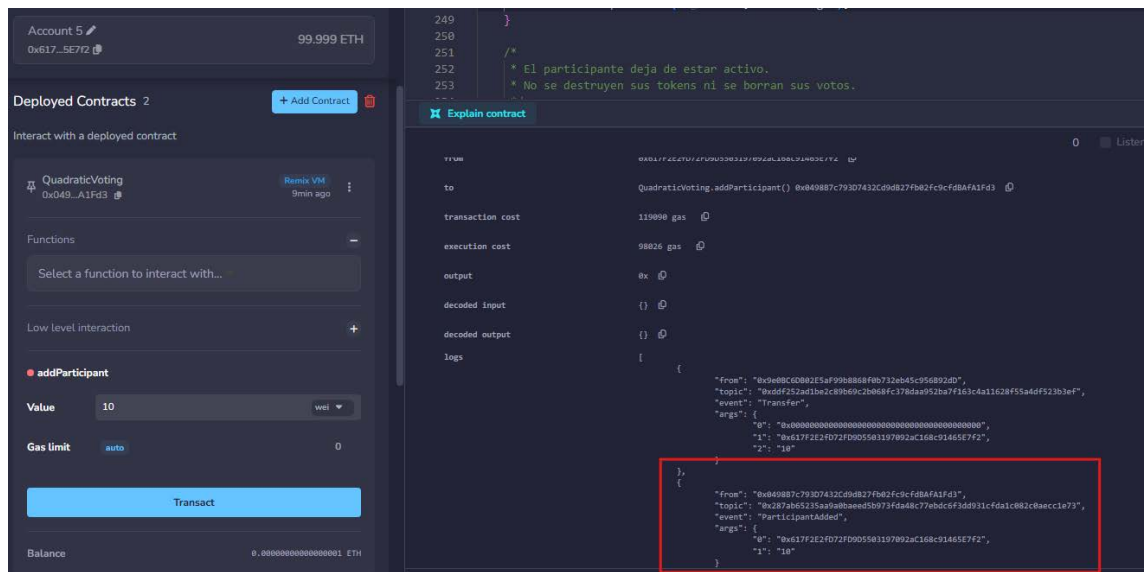
El resultado fue correcto: QuadraticVoting quedó desplegado, VotingToken fue creado automáticamente, y la dirección devuelta por `getERC20()` permitió interactuar con el token ERC20 del sistema.



4.2. Registro de participantes

A continuación, se probó el registro de participantes mediante la función `addParticipant()`. Para ello, se utilizaron distintas cuentas de Remix, cada una enviando una cantidad de Ether al contrato. Ese Ether se utilizó para comprar tokens en el momento de la inscripción.

Después de cada registro, se comprobó el balance del participante en el contrato VotingToken mediante la función `balanceOf()`. Esta comprobación permite verificar que los tokens se han creado y asignado correctamente al usuario registrado.



4.3. Compra, venta y eliminación de participante

También se comprobó la compra adicional de tokens por parte de un participante ya registrado mediante la función `buyTokens()`. Esta función permite enviar más Ether al contrato para recibir nuevos tokens de votación.

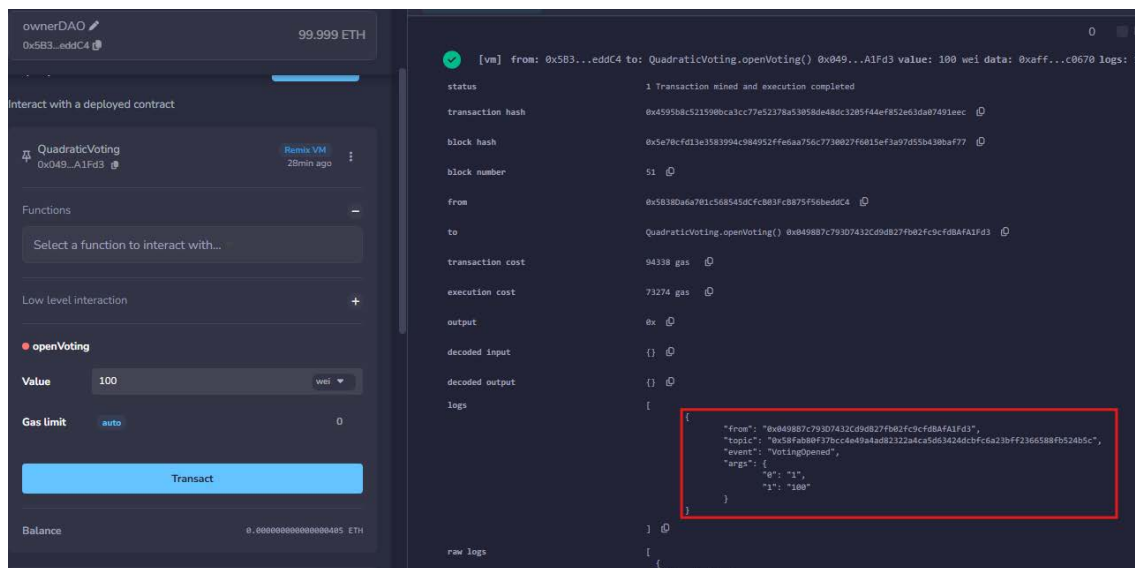
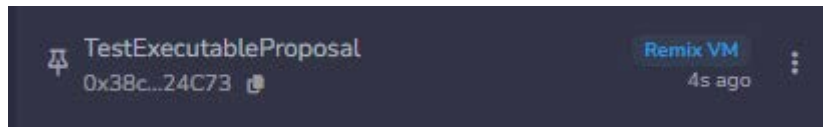
Después se probó la operación inversa mediante `sellTokens()`. En esta función, el participante devuelve tokens no bloqueados y recupera el Ether correspondiente. Internamente, los tokens vendidos son destruidos mediante `burn()`, y el contrato transfiere al usuario la cantidad de Ether equivalente, siempre que exista reserva disponible fuera del presupuesto de votación.

Finalmente, se probó `removeParticipant()`, que permite a un participante darse de baja del sistema. Tras ejecutar esta función, el usuario deja de poder crear propuestas, votar, comprar o vender tokens a través de QuadraticVoting, salvo que vuelva a registrarse.

El resultado fue el esperado: el participante pudo comprar tokens, vender tokens no bloqueados y eliminarse correctamente del sistema.

The screenshot shows a web interface on the left and transaction details on the right. On the left, under 'Low level interaction', the 'sellTokens' function is selected with a value of 5. The 'Transact' button is visible. On the right, the transaction details for block 45 are shown. The 'decoded output' is an empty array, and the 'logs' section contains two entries. The first log is a 'Transfer' event, and the second log is a 'TokensSold' event, both with arguments indicating the transaction details.

The screenshot shows a web interface for the 'VotingToken' contract. Under 'Low level interaction', the 'balanceOf' function is selected with the address '0x617F2E2fD72FD9D5503197092aC168c91465E7f2'. The 'Call' button is visible. The 'Balance' section at the bottom shows '0 ETH'.

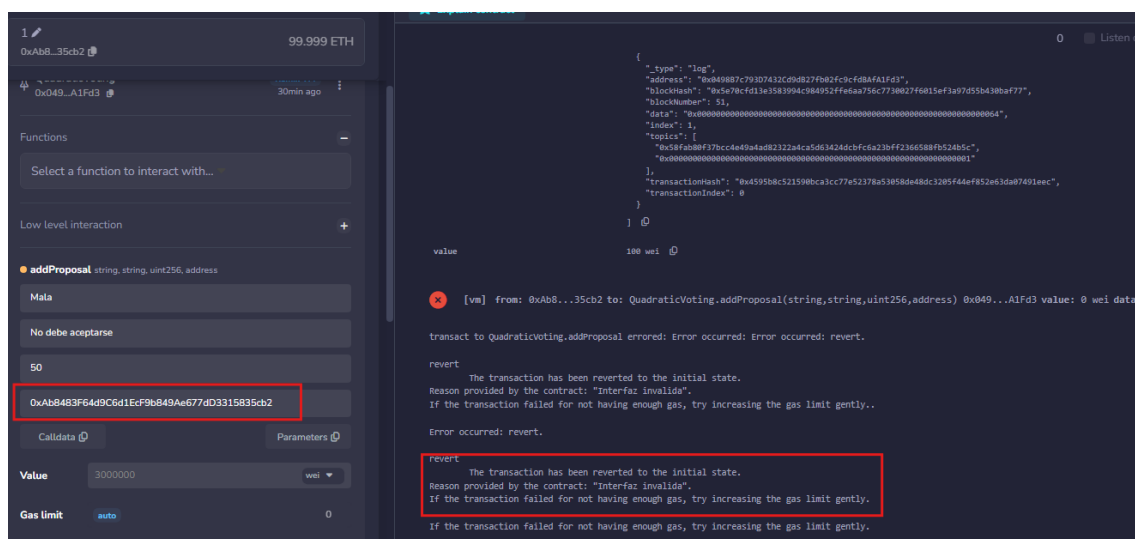


4.5. Creación de propuesta inválida

Antes de crear una propuesta válida, se probó el comportamiento del contrato ante una dirección de propuesta incorrecta. Para ello, se llamó a `addProposal()` utilizando como contrato externo una cuenta normal o una dirección que no implementaba `IExecutableProposal`.

Esta prueba es importante porque el contrato debe rechazar propuestas cuyo contrato externo no implemente la interfaz exigida. De esta forma, se evita aceptar propuestas que después no podrían ejecutarse correctamente.

El resultado fue el esperado: la transacción revirtió y el contrato no aceptó la propuesta. Esto demuestra que la comprobación mediante ERC165 funciona correctamente.

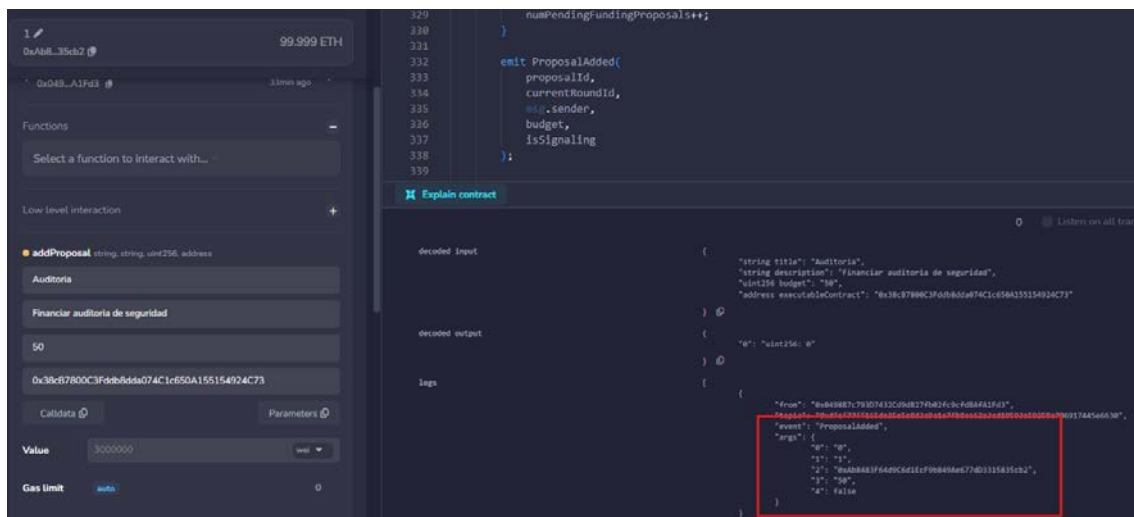


4.6. Creación de propuesta de financiación válida

A continuación, se desplegó el contrato auxiliar TestExecutableProposal y se utilizó su dirección como contrato externo válido. Después, se creó una propuesta de financiación mediante addProposal(), indicando un título, una descripción, un presupuesto mayor que cero y la dirección del contrato externo.

Al tener un presupuesto superior a cero, la propuesta fue clasificada como propuesta de financiación. Por tanto, quedó registrada como pendiente y se incrementó numPendingFundingProposals.

El resultado fue correcto: la propuesta quedó registrada, fue clasificada como propuesta de financiación y pasó a formar parte del conjunto de propuestas pendientes.

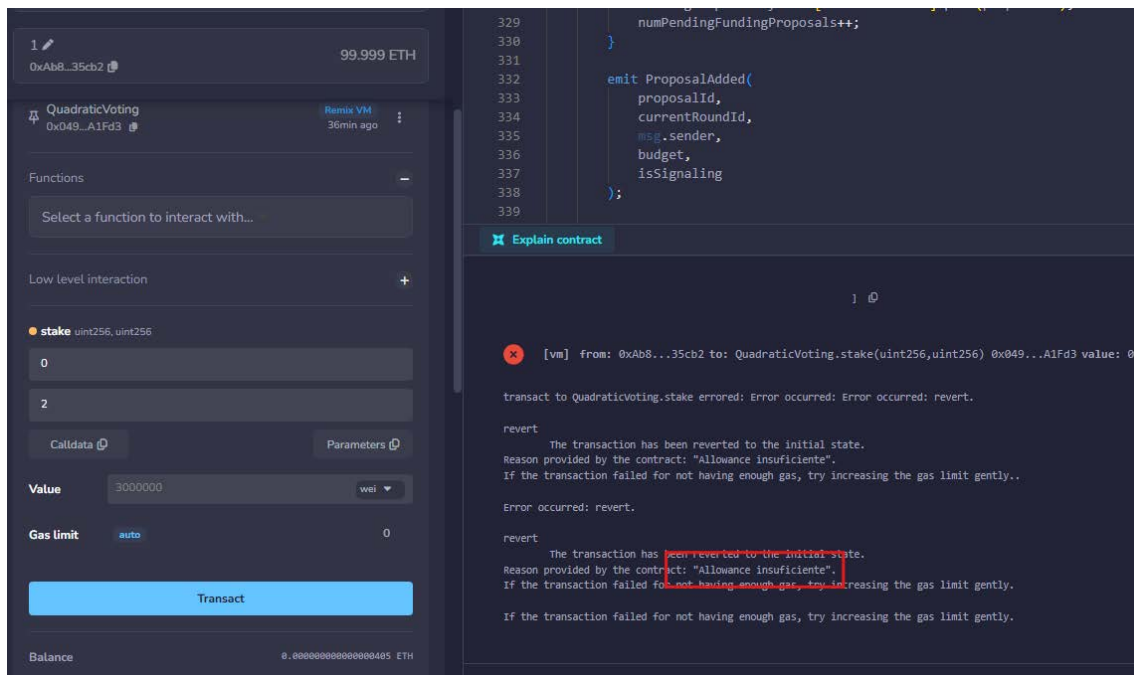


4.7. Prueba negativa: votar sin approve

Antes de realizar una votación válida, se intentó ejecutar stake() sin haber llamado previamente a approve() en el contrato ERC20. Esta prueba verifica que QuadraticVoting no puede mover tokens de un usuario sin autorización expresa.

La transacción debía revertir porque la autorización (allowance) era insuficiente para transferir los tokens necesarios al contrato de votación.

El resultado fue el esperado: la transacción falló por allowance insuficiente. Esto confirma que el contrato respeta el modelo ERC20 y no puede bloquear tokens sin permiso del propietario.

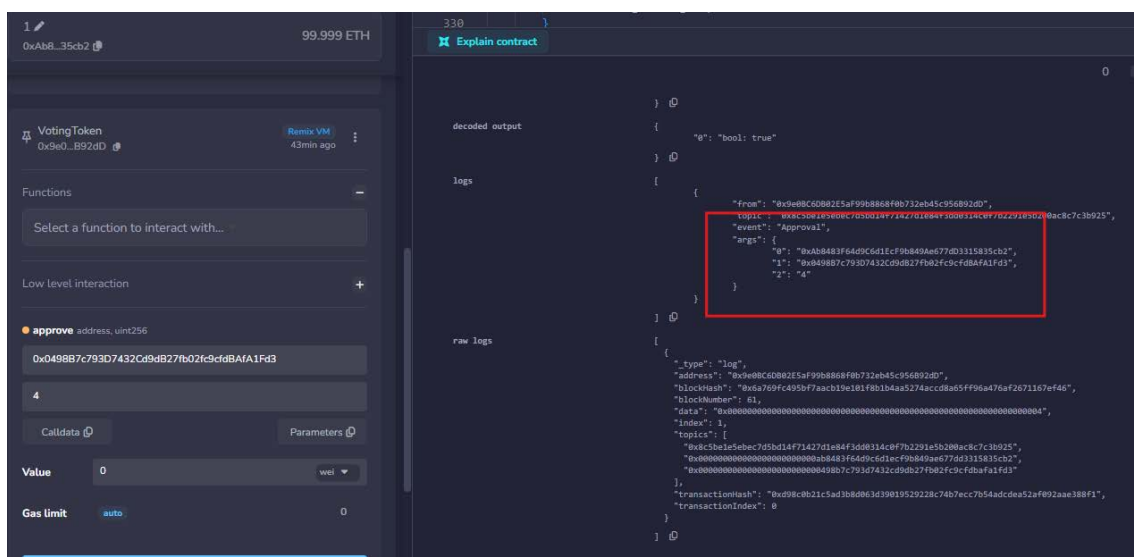


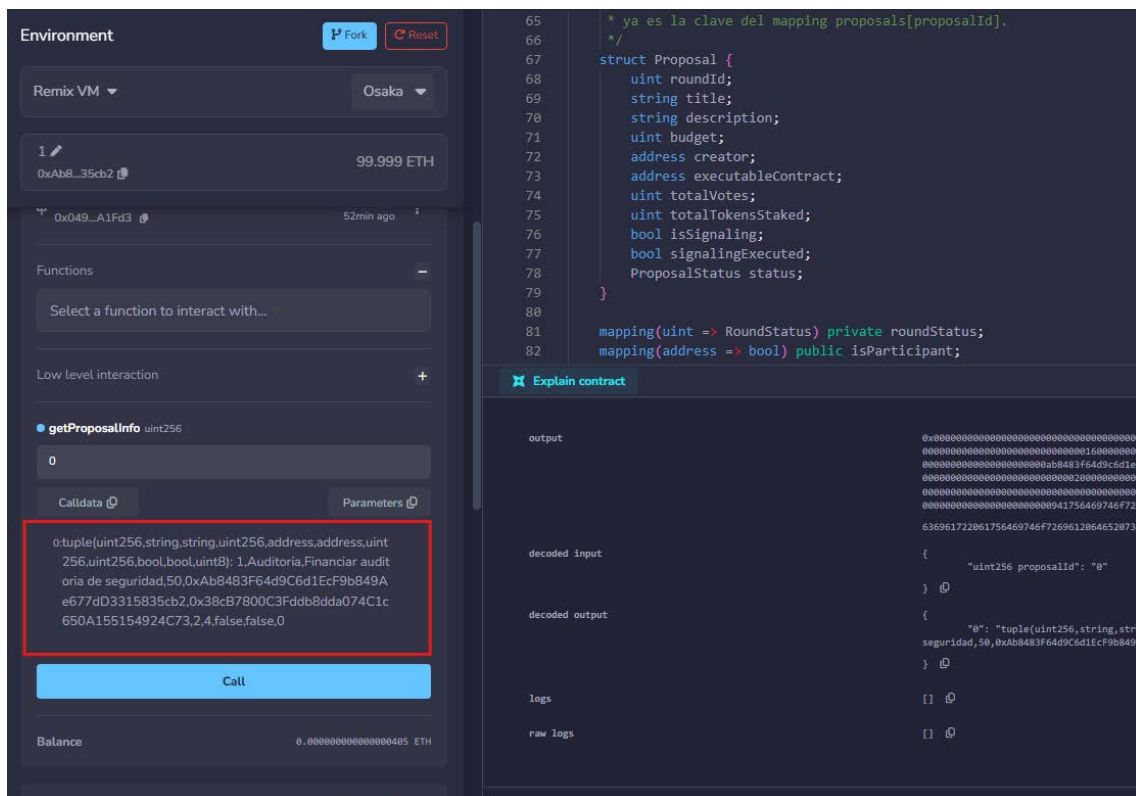
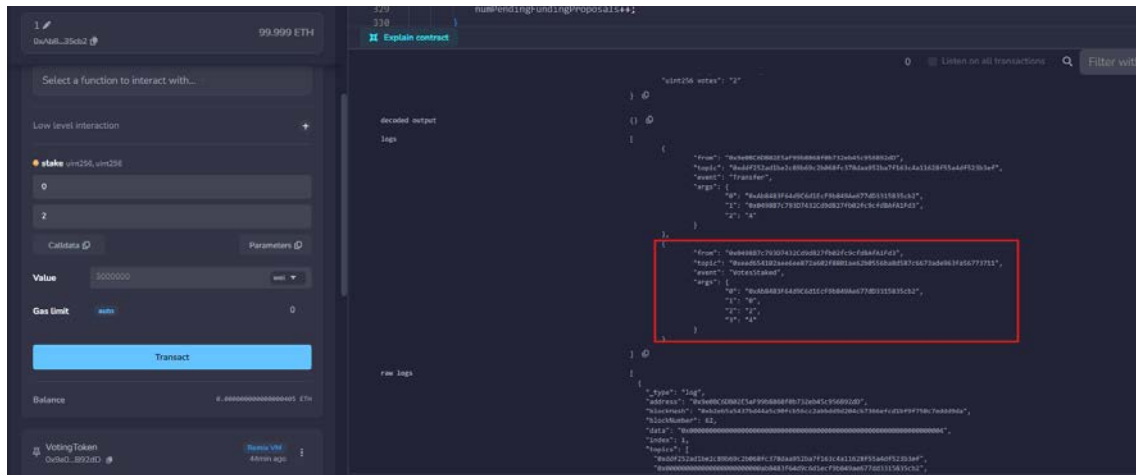
4.8. Aprobación de tokens y votación con stake

Tras comprobar el caso negativo anterior, el participante ejecutó `approve()` en `VotingToken`, autorizando a `QuadraticVoting` a gastar los tokens necesarios para votar. Después, se ejecutó `stake()` sobre la propuesta correspondiente.

En la prueba realizada, el usuario depositó dos votos sobre una propuesta en la que no tenía votos previos. Por tanto, el coste cuadrático fue de cuatro tokens, ya que el coste total de dos votos es $2^2 = 4$.

El resultado fue correcto: el contrato comprobó la autorización, ejecutó `transferFrom()`, bloqueó los tokens en `QuadraticVoting` y actualizó los votos y tokens asociados a la propuesta.



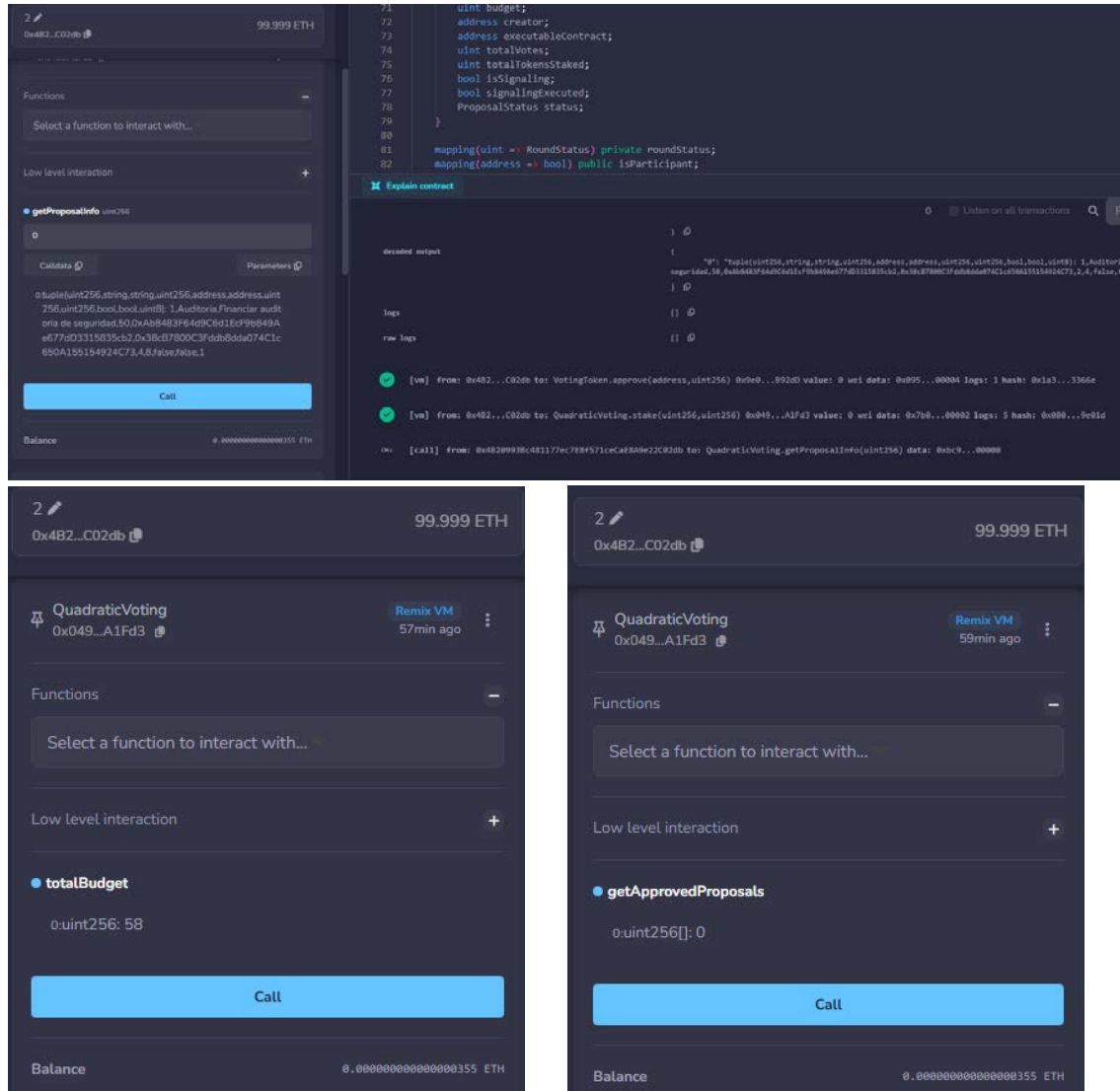


4.9. Aprobación automática de una propuesta de financiación

Se continuó votando la propuesta de financiación hasta superar el umbral de aprobación. Cuando se cumplieron simultáneamente las dos condiciones exigidas, presupuesto suficiente y número de votos superior al umbral, la propuesta fue aprobada automáticamente.

Al aprobarse, el contrato ejecutó internamente `_checkAndExecuteProposal()`. Esta función marcó la propuesta como aprobada, actualizó el presupuesto disponible, consumió los tokens utilizados en la votación y ejecutó el contrato externo mediante `executeProposal()`, enviando el Ether presupuestado.

El resultado fue correcto: la propuesta se aprobó automáticamente, el contrato externo recibió el Ether correspondiente, los tokens usados en la votación se consumieron y el presupuesto se actualizó correctamente.

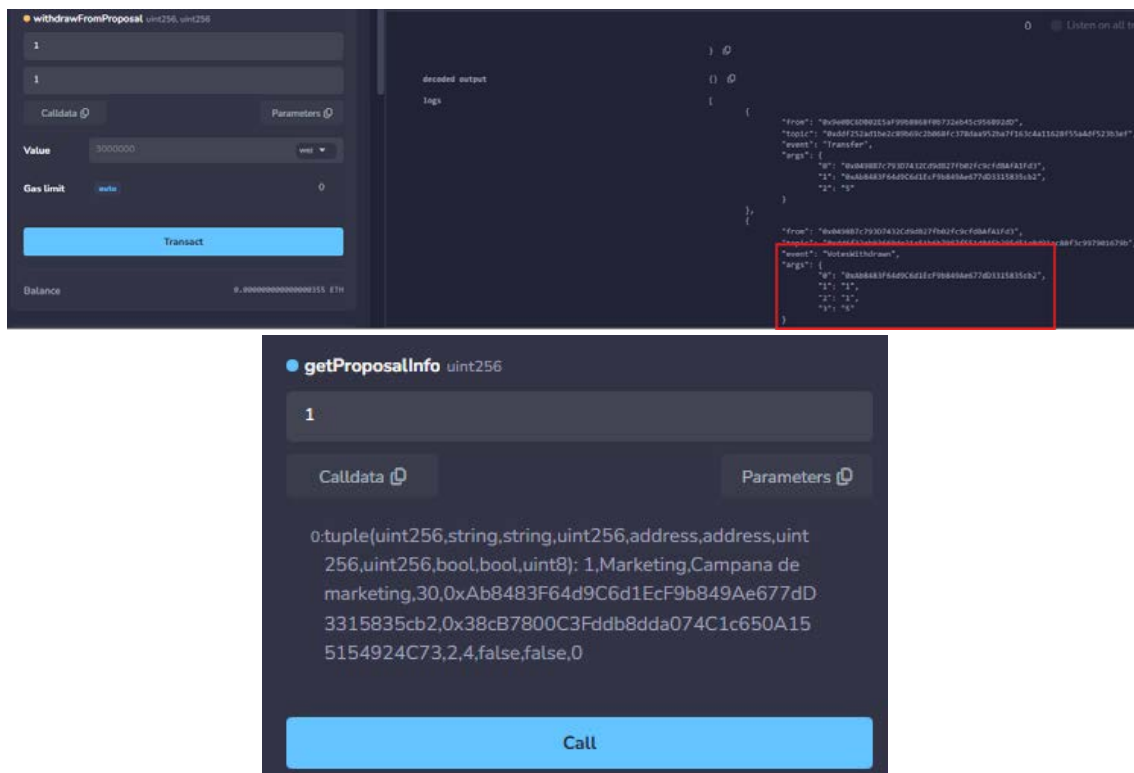


4.10. Retirada de votos

Se probó la retirada de votos mediante `withdrawFromProposal()` sobre una propuesta que seguía pendiente. Esta función permite que un participante retire votos propios siempre que la propuesta no haya sido aprobada ni cancelada.

En la prueba realizada, un usuario pasó de tres votos a dos votos. El coste anterior era de nueve tokens, ya que $3^2 = 9$, mientras que el nuevo coste era de cuatro tokens, ya que $2^2 = 4$. Por tanto, el contrato debía devolver cinco tokens al participante.

El resultado fue correcto: solo se retiraron votos propios, la propuesta seguía pendiente y el contrato devolvió la diferencia cuadrática correspondiente.



4.11. Cancelación de propuesta y reembolso

También se probó la cancelación de una propuesta mediante `cancelProposal()`. Para ello, primero se creó una propuesta, se depositaron votos en ella y posteriormente su creador la canceló.

Dado que el sistema implementa pull-over-push, la cancelación no devuelve automáticamente los tokens a los votantes. En su lugar, la propuesta queda marcada como cancelada y cada usuario puede reclamar posteriormente sus tokens mediante `claimRefund()`.

El resultado fue el esperado: solo el creador pudo cancelar la propuesta, no se permitió cancelar una propuesta aprobada y los tokens se recuperaron correctamente mediante `claimRefund()`.

```

[vm] from: 0x4B2...C02db to: QuadraticVoting.addProposal(string,string,uint256,address) 0x049...A1Fd3 value: 0 wei data: 0x9ab...00000 logs: 1 hash: 0xd5f...69612

[vm] from: 0x4B2...C02db to: VotingToken.approve(address,uint256) 0x9e0...B92d0 value: 0 wei data: 0x095...00001 logs: 1 hash: 0x30d...8db67

[vm] from: 0x4B2...C02db to: QuadraticVoting.stake(uint256,uint256) 0x049...A1Fd3 value: 0 wei data: 0x7b0...00001 logs: 2 hash: 0x9da...52b42

[vm] from: 0x4B2...C02db to: QuadraticVoting.cancelProposal(uint256) 0x049...A1Fd3 value: 0 wei data: 0xe0a...00002 logs: 1 hash: 0xf34...daac5

status      1 Transaction mined and execution completed
transaction hash  0xf34b3ef671d92333acb989d2fff7499f7bcb11176aa5f58a4130f0075daac5
block hash      0xb17db49a5ea980c78af1bc564fd8f544849915e2ae4511cccb427dc96baedf2b
block number    72
from           0x48209938c481177ec7E8f571ceCaE8A9e22C02db
to             QuadraticVoting.cancelProposal(uint256) 0x049887c79307432Cd9d827fb02fc9cfdbAFA1Fd3
transaction cost 61627 gas
execution cost  40423 gas
output         0x
decoded input   {
  "uint256 proposalId": "2"
}
decoded output  {}
logs           [
  {
    "from": "0x049887c79307432Cd9d827fb02fc9cfdbAFA1Fd3",
    "topic": "0xd16e669c63d9a3a5e36ee7cc7e2104b8db28ccd286aa18966e98fa230c73b08c",
    "event": "ProposalCancelled",
    "args": {
      "0": "2"
    }
  }
]

[vm] from: 0x4B2...C02db to: QuadraticVoting.claimRefund(uint256) 0x049...A1Fd3 value: 0 wei data: 0x5b7...00002 logs: 2 hash: 0x135...3bfa8

status      1 Transaction mined and execution completed
transaction hash  0x13559b01e8753263aa152eb9ee5bb7d078e14694ada4dc21d996f06c87b3bfa8
block hash      0x874c24573747eba40a398c1f0155cd287b0ad9c65f0db149b9601b9a8fb3d30d
block number    73
from           0x48209938c481177ec7E8f571ceCaE8A9e22C02db
to             QuadraticVoting.claimRefund(uint256) 0x049887c79307432Cd9d827fb02fc9cfdbAFA1Fd3
transaction cost 80582 gas
execution cost  79523 gas
output         0x
decoded input   {
  "uint256 proposalId": "2"
}
decoded output  {}
logs           [
  {
    "from": "0x049887c79307432Cd9d827fb02fc9cfdbAFA1Fd3",
    "topic": "0xddf252ad1be2c89b69c2b068fc378daa952ba7f163c4a11628f55a4df523b3ef",
    "event": "Transfer",
    "args": {
      "0": "0x049887c79307432Cd9d827fb02fc9cfdbAFA1Fd3",
      "1": "0x48209938c481177ec7E8f571ceCaE8A9e22C02db",
      "2": "1"
    }
  },
  {
    "from": "0x049887c79307432Cd9d827fb02fc9cfdbAFA1Fd3",
    "topic": "0x17c1089037710a4991a771a0b53802a1e7f51590c6c9307000d1300530",
    "event": "RefundClaimed",
    "args": {
      "0": "0x48209938c481177ec7E8f571ceCaE8A9e22C02db",
      "1": "2",
      "2": "1"
    }
  }
]

```

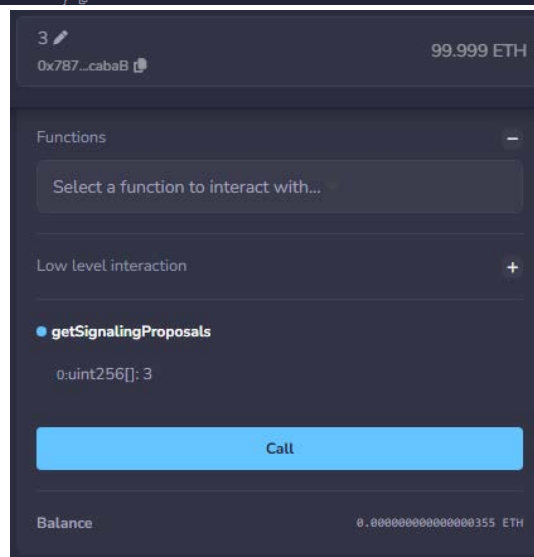
4.12. Creación y votación de propuesta de signaling

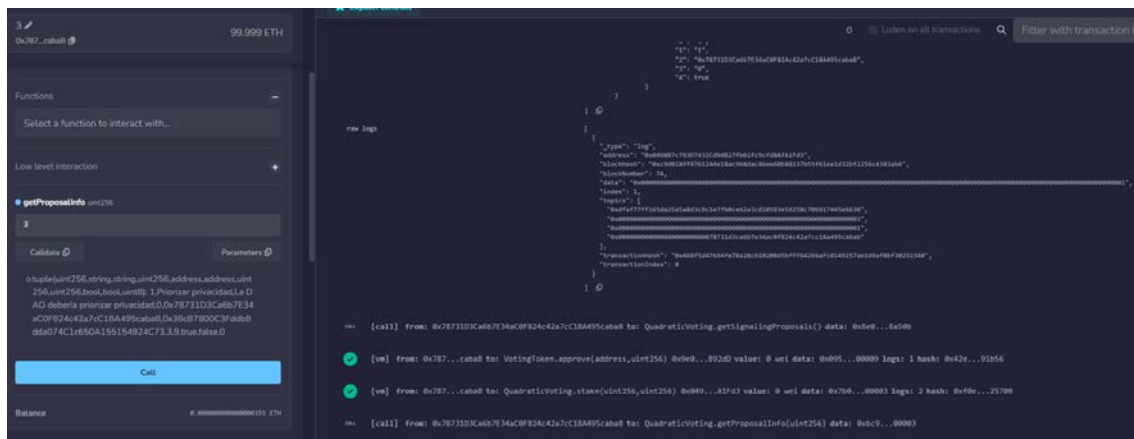
Se creó una propuesta de signaling mediante `addProposal()` indicando un presupuesto igual a cero. Esta condición permite diferenciarla de las propuestas de financiación.

Después, la propuesta fue votada mediante `stake()`. Aunque los tokens se bloquearon correctamente, la propuesta no fue aprobada por umbral ni ejecutada durante la votación, ya que las propuestas de signaling solo se ejecutan después del cierre de la ronda.

El resultado fue correcto: la propuesta se registró como signaling, no aumentó numPendingFundingProposals, pudo recibir votos y no se ejecutó automáticamente.

```
[vm] from: 0x787...caba8 to: QuadraticVoting.addProposal(string,string,uint256,address) 0x049...A1fd3 value: 0 wei data: 0x9ab...00000 logs: 1 hash: 0x468...92340
status 1 Transaction mined and execution completed
transaction hash 0x468f5d47694fe78a28c610288d5bfff64266afc0149257ae1d9af0bf30292340 ⓘ
block hash 0xc9d018ff9761244e18ac968dac86ee60b88137b55f61eed32b2f256c4383ab6 ⓘ
block number 74 ⓘ
from 0x78731D3Ca6b7E34aC8F824c42a7cC18A495caba8 ⓘ
to QuadraticVoting.addProposal(string,string,uint256,address) 0x049887c79307432Cd9d827Fb02fc9cfdbAFA1fd3 ⓘ
transaction cost 274367 gas ⓘ
execution cost 251215 gas ⓘ
output 0x0000000000000000000000000000000000000000000000000000000000000000 ⓘ
decoded input {
  "string title": "Priorizar privacidad",
  "string description": "La DAO deberia priorizar privacidad",
  "uint256 budget": "0",
  "address executableContract": "0x38c87808c3fdd88da0741c650A155154924C73"
} ⓘ
decoded output {
  "0": "uint256: 3"
} ⓘ
```

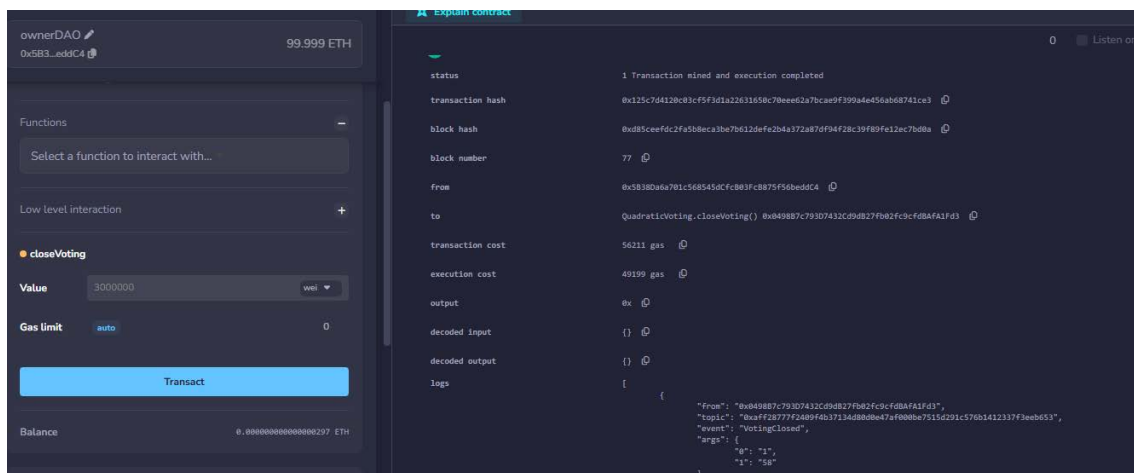




4.13. Cierre de votación

El owner cerró la ronda mediante `closeVoting()`. En el diseño implementado, esta función no recorre todos los votantes ni ejecuta todas las propuestas de signaling. Únicamente marca la ronda como cerrada, pone a cero el presupuesto activo y transfiere al owner el presupuesto no gastado.

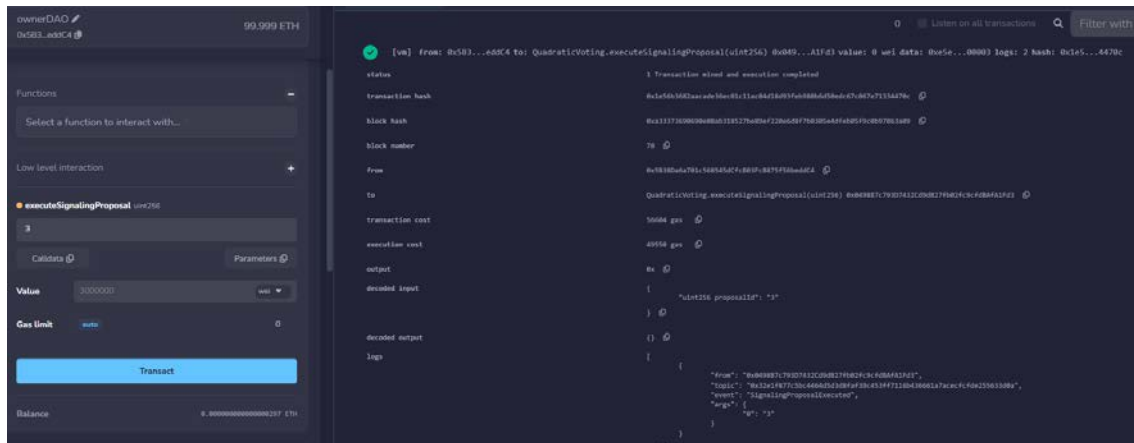
El resultado fue correcto: la ronda quedó cerrada, dejaron de aceptarse nuevas propuestas y votos, el presupuesto restante fue transferido al owner y las operaciones pendientes quedaron disponibles bajo demanda.



4.14. Ejecución de propuesta de signaling después del cierre

Tras cerrar la votación, se ejecutó la propuesta de signaling mediante `executeSignalingProposal()`. Esta función llamó al contrato externo asociado sin enviar Ether, ya que las propuestas de signaling tienen presupuesto cero.

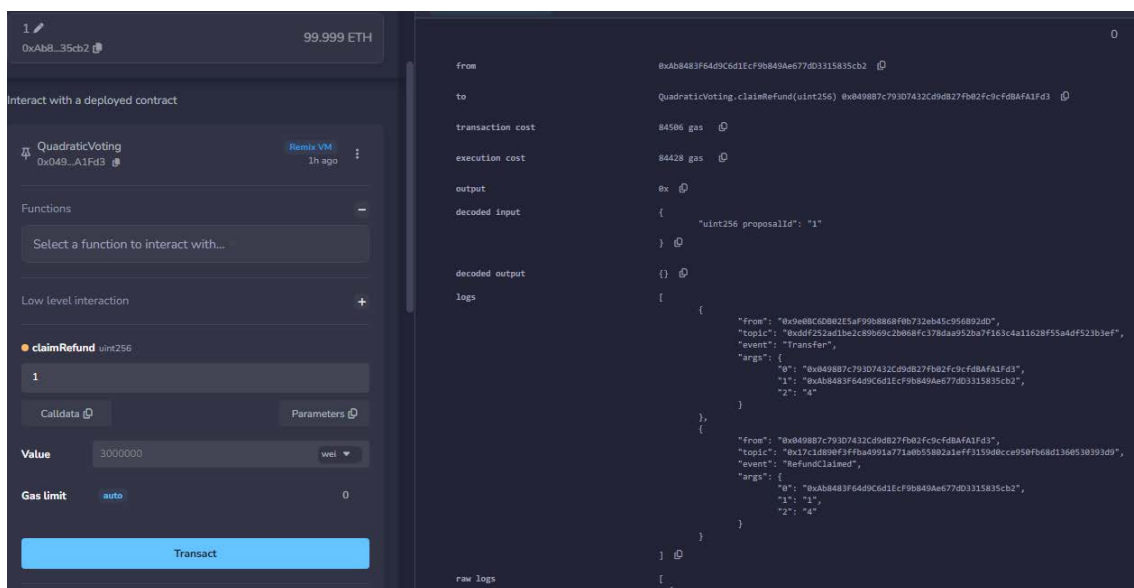
El resultado fue correcto: la propuesta de signaling se ejecutó después del cierre, no se transfirió Ether y el contrato impidió que pudiera ejecutarse más de una vez.



4.15. Reclamación de tokens de propuesta no aprobada

Una propuesta de financiación que no alcanzó el umbral antes del cierre quedó en estado pendiente. Después de cerrar la votación, el participante reclamó los tokens bloqueados en esa propuesta mediante `claimRefund()`.

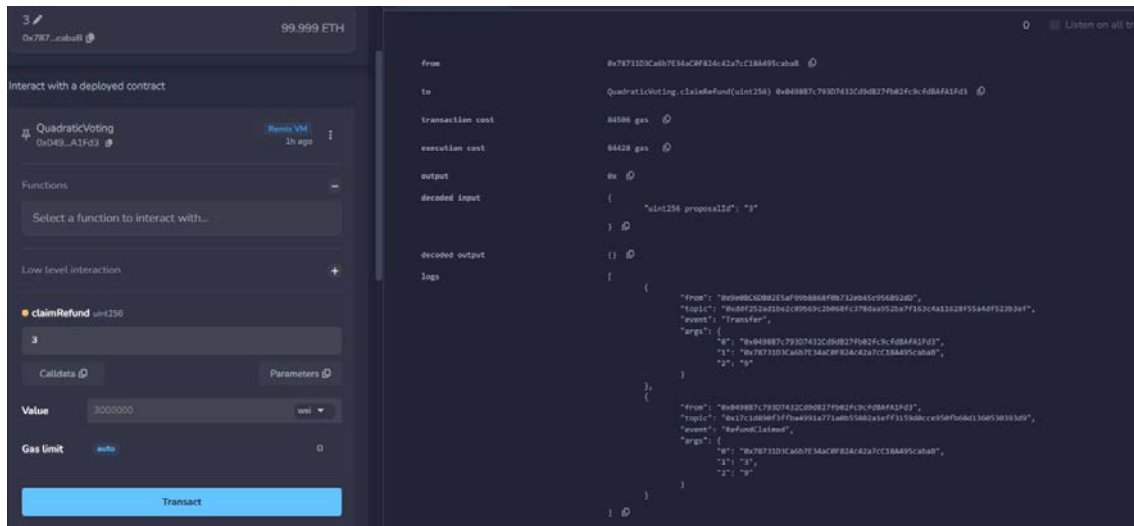
El resultado fue correcto: los tokens de la propuesta no aprobada se devolvieron al participante y la devolución se realizó bajo demanda.



4.16. Reclamación de tokens de propuesta de signaling

También se probó la devolución de tokens bloqueados en una propuesta de signaling. Después del cierre de la ronda, el participante ejecutó `claimRefund()` sobre dicha propuesta.

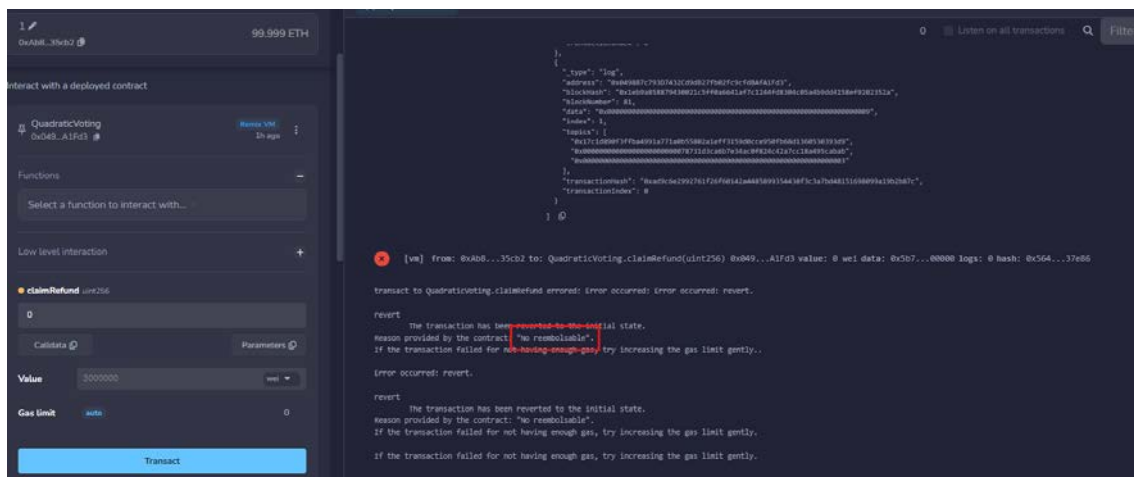
El resultado fue correcto: los tokens utilizados para votar la propuesta de signaling se devolvieron después del cierre.



4.17. Prueba negativa: no se puede reclamar en propuesta aprobada

Finalmente, se realizó una prueba negativa intentando ejecutar `claimRefund()` sobre una propuesta de financiación ya aprobada. Esta operación debía fallar, porque los tokens utilizados en propuestas aprobadas se consumen y no son reembolsables.

El resultado fue el esperado: la transacción revirtió, demostrando que el contrato distingue correctamente entre propuestas aprobadas, canceladas, no aprobadas y de signaling.



5. Seguridad y análisis de vulnerabilidades

El sistema desarrollado maneja Ether y tokens ERC20 con valor económico, por lo que la seguridad es una parte esencial del diseño. Además, el contrato `QuadraticVoting` no solo gestiona fondos propios, sino que también interactúa con contratos externos de propuesta mediante `executeProposal()`. Por este motivo, se han tenido en cuenta distintas vulnerabilidades propias de los smart contracts, prestando especial atención a los ataques de reentrada, al control de permisos, a la protección de los tokens y a la ejecución segura de contratos externos.

5.1. Protección frente a reentrancy

La vulnerabilidad más relevante en este proyecto es la reentrancy. Este tipo de ataque aparece cuando un contrato realiza una llamada externa antes de haber terminado correctamente la actualización de su estado interno. Si el destinatario de esa llamada es un contrato malicioso, podría intentar volver a entrar en la misma función, o en otra función sensible, antes de que la primera ejecución haya finalizado.

En este proyecto existen varias situaciones especialmente delicadas desde el punto de vista de la reentrada. Por ejemplo, `sellTokens()` envía Ether al participante cuando vende tokens; `closeVoting()` transfiere al owner el presupuesto no gastado; `_checkAndExecuteProposal()` ejecuta un contrato externo de propuesta; `claimRefund()` devuelve tokens bloqueados; y `executeSignalingProposal()` también llama a un contrato externo. Todas estas operaciones implican una interacción con direcciones externas y, por tanto, deben estar protegidas.

Para mitigar este riesgo, se ha implementado un modificador `nonReentrant` basado en una variable booleana `locked`. Su función es impedir que una función protegida pueda volver a ejecutarse mientras ya se encuentra en curso.

```
modifier nonReentrant() {  
    require(!locked, "Reentrancy");  
    locked = true;  
    _;  
    locked = false;  
}
```

El funcionamiento es sencillo. Antes de ejecutar el cuerpo de la función, el contrato comprueba que `locked` sea `false`. Si la función empieza a ejecutarse correctamente, `locked` pasa a valer `true`. Mientras esta variable esté activa, cualquier intento de reentrar en otra función protegida por `nonReentrant` fallará. Al terminar la ejecución normal de la función, `locked` vuelve a `false`.

Además del modificador `nonReentrant`, el contrato sigue el patrón `checks-effects-interactions`. Esto significa que primero se comprueban las condiciones de ejecución, después se actualiza el estado interno del contrato y, solo al final, se realizan llamadas externas o transferencias. Este orden es fundamental para evitar que un contrato externo encuentre el sistema en un estado incoherente.

Un ejemplo claro aparece en `closeVoting()`. Antes de transferir el presupuesto sobrante al owner, el contrato marca la ronda como cerrada, pone `totalBudget` a cero y reinicia el contador de propuestas pendientes. Solo después realiza la llamada externa con `call`. De esta forma, incluso si el destinatario fuese un contrato

con código malicioso, el estado principal de la votación ya habría quedado actualizado antes de la interacción externa.

También ocurre algo similar en la ejecución de propuestas de financiación. Antes de llamar a `executeProposal()` sobre el contrato externo, el contrato marca la propuesta como aprobada, actualiza el presupuesto disponible, añade la propuesta al conjunto de aprobadas y destruye los tokens consumidos. Por tanto, la propuesta externa no se ejecuta mientras el contrato de votación sigue en un estado anterior a la aprobación.

En definitiva, la protección frente a reentrancy se basa en dos ideas complementarias: impedir reentradas mediante `nonReentrant` y ordenar correctamente la lógica interna mediante `checks-effects-interactions`. Esta combinación resulta especialmente importante en este proyecto porque se manejan Ether, tokens ERC20 y llamadas a contratos externos potencialmente no confiables.

5.2. Contratos externos maliciosos

Otro riesgo relevante es la ejecución de contratos externos de propuesta. El sistema recibe una dirección en `addProposal()` y, si la propuesta es aprobada, llama a `executeProposal()` sobre esa dirección. Esto implica que el contrato externo podría ser malicioso, revertir, intentar consumir demasiado gas o tratar de reentrar en el sistema.

Para reducir este riesgo, el contrato comprueba mediante ERC165 que la dirección proporcionada implementa la interfaz `IExecutableProposal`. Además, la llamada externa se realiza con un límite máximo de gas de 100000, evitando que el contrato de propuesta pueda consumir todo el gas de la transacción. Por último, como se ha explicado en el apartado anterior, el estado interno se actualiza antes de ejecutar la llamada externa y las funciones sensibles están protegidas con `nonReentrant`.

5.3. Denegación de servicio por consumo de gas

También se ha tenido en cuenta el riesgo de denegación de servicio por consumo excesivo de gas. Este problema podría aparecer si `closeVoting()` intentase recorrer todas las propuestas y todos los votantes para devolver tokens y ejecutar propuestas de signaling.

Para evitarlo, se ha implementado el patrón `pull-over-push`. En lugar de realizar todas las devoluciones dentro de `closeVoting()`, cada usuario reclama sus tokens individualmente mediante `claimRefund()`. Del mismo modo, las propuestas de signaling se ejecutan de forma individual mediante `executeSignalingProposal()`. Así, el cierre de la votación no depende de recorrer estructuras de datos potencialmente grandes.

5.4. Protección de los tokens ERC20

La manipulación de tokens es otro punto crítico. Para evitar que un usuario pueda crear o destruir tokens arbitrariamente, las funciones `mint()` y `burn()` de `VotingToken` solo pueden ser ejecutadas por el contrato `QuadraticVoting`.

Además, el contrato principal no puede mover tokens de un participante sin autorización previa. Antes de votar, el usuario debe ejecutar `approve()` en el contrato ERC20, autorizando a `QuadraticVoting` a gastar una cantidad concreta de tokens. Después, `stake()` comprueba la autorización mediante `allowance()` y mueve los tokens con `transferFrom()`. De esta forma, se respeta la propiedad de los tokens y se evita cualquier transferencia no autorizada.

5.5. Transferencias de Ether con `call`

Las transferencias de Ether se realizan con `call` en lugar de `transfer` o `send`. Esta decisión evita el límite de 2300 unidades de gas impuesto por `transfer` y `send`, permitiendo que el receptor pueda ser también un contrato. En todos los casos se comprueba el valor de retorno de la llamada mediante `require(success)`, de forma que si el envío falla, la transacción revierte.

Esta técnica se utiliza, por ejemplo, al devolver el presupuesto restante al owner en `closeVoting()` y al devolver Ether a un participante en `sellTokens()`.

5.6. Control de permisos y validación de estados

El contrato utiliza modificadores y comprobaciones de estado para impedir operaciones no autorizadas. El modificador `onlyOwner` restringe la apertura y cierre de votaciones al creador del contrato. El modificador `onlyParticipant` limita funciones como `buyTokens()`, `sellTokens()`, `addProposal()`, `stake()` y `withdrawFromProposal()` a usuarios registrados.

Además, las rondas y propuestas tienen estados explícitos. Las rondas pueden estar en `NotStarted`, `Open` o `Closed`, y las propuestas pueden estar en `Pending`, `Approved` o `Cancelled`. Esto permite evitar operaciones inválidas, como votar con la ronda cerrada, cancelar una propuesta aprobada, ejecutar dos veces una propuesta de signaling o reclamar tokens de una propuesta ya aprobada.

5.7. Tamaño del bytecode

Durante el desarrollo también se tuvo en cuenta el tamaño del bytecode generado por el contrato. `QuadraticVoting` concentra mucha lógica: gestión de participantes, tokens, propuestas, votación cuadrática, ejecución externa, seguridad y patrón `pull-over-push`. Para mantener el contrato dentro de un tamaño razonable, se redujeron mensajes largos de `require`, se evitaron getters innecesarios y se simplificaron algunas funciones auxiliares sin eliminar funcionalidad esencial.

5.8. Conclusión del análisis de seguridad

La principal amenaza del sistema es la reentrancy, debido a que el contrato realiza transferencias de Ether, devuelve tokens y ejecuta contratos externos de propuesta. Para mitigarla, se ha utilizado un modificador `nonReentrant` y se ha seguido el patrón `checks-effects-interactions` en las funciones críticas.

El resto de las medidas complementan esta protección principal: ERC165 evita aceptar contratos externos incompatibles, el límite de gas reduce el riesgo de abuso en `executeProposal()`, el patrón `pull-over-push` evita bloqueos por consumo de gas, y el uso de `approve/transferFrom` garantiza que los tokens no se mueven sin autorización del propietario. En conjunto, estas decisiones permiten que el sistema sea más seguro y defendible frente a los ataques más relevantes tratados en la asignatura.

6. Conclusiones

El sistema desarrollado implementa un mecanismo completo de votación cuadrática on-chain para DAOs. Permite registrar participantes, comprar y vender tokens ERC20, crear propuestas de financiación y de signaling, votar con coste cuadrático, aprobar automáticamente propuestas de financiación, ejecutar contratos externos y devolver tokens en los casos correspondientes.

El diseño se ha planteado siguiendo los requisitos del enunciado, utilizando `QuadraticVoting` como contrato principal, `VotingToken` como token ERC20 basado en `OpenZeppelin`, `IExecutableProposal` como interfaz obligatoria para propuestas externas y `TestExecutableProposal` como contrato auxiliar de prueba.

Uno de los aspectos más relevantes del proyecto es la integración del patrón `pull-over-push`. Esta decisión mejora la escalabilidad y seguridad del cierre de votación, evitando que `closeVoting()` tenga que recorrer todos los votantes o todas las propuestas. En su lugar, las devoluciones de tokens y la ejecución de propuestas de signaling se realizan bajo demanda.

Desde el punto de vista de seguridad, el contrato incorpora mecanismos para proteger la gestión de Ether y tokens, incluyendo `approve` y `transferFrom`, restricción de `mint` y `burn`, verificación ERC165, uso de `call` para transferencias de Ether, limitación de gas en llamadas externas y protección frente a reentrancy.

Las pruebas realizadas en Remix cubren los flujos principales del sistema y también varios casos negativos: creación de propuestas inválidas, intento de votar sin `approve`, reclamación de tokens en propuestas aprobadas y validación de ejecución externa. Estas pruebas permiten comprobar que el sistema se comporta correctamente y que las restricciones implementadas funcionan como se esperaba.

En conjunto, el proyecto cumple los requisitos funcionales establecidos y presenta un diseño razonablemente seguro, mantenible y defendible para un sistema de gobernanza on-chain basado en votación cuadrática.